



Segurança com Spring Security

Programação Web II

Segurança de Aplicações Web

O que é segurança de aplicações web?

A segurança de aplicações web envolve práticas e tecnologias usadas para proteger aplicações contra ameaças e vulnerabilidades, garantindo a integridade, a confidencialidade e a disponibilidade dos dados.

OWASP e práticas Secure by Design

A **OWASP** Top 10 é uma lista das 10 vulnerabilidades mais críticas em aplicações web, publicada pela organização sem fins lucrativos *OWASP* (*Open Web Application Security Project*).

Essa lista serve como um guia essencial para desenvolvedores e equipes de segurança, ajudando-os a identificar, entender e mitigar riscos em suas aplicações.

Alguns exemplos comuns incluem falhas como *Injeção de SQL*, *Cross-Site Scripting (XSS)* e *Controle de Acesso Quebrado*, que podem comprometer a segurança de dados e a integridade das aplicações se não forem tratados adequadamente.

Leia mais: [OWASP Top 10](#)

OWASP e práticas Secure by Design

Secure by Design é uma abordagem de desenvolvimento de software que prioriza a segurança desde as primeiras etapas do projeto.

Em vez de adicionar camadas de proteção posteriormente, a segurança é incorporada ao design e ao desenvolvimento do sistema, garantindo que ele seja seguro por padrão.

Essa prática envolve **validar dados de entrada**, **proteger informações sensíveis**, **aplicar controle de acesso rigoroso** e **realizar auditorias contínuas** para identificar e mitigar vulnerabilidades potenciais.

OWASP e práticas Secure by Design

Práticas Secure by Design

- **Validação de entrada:** sanitizar e validar todos os dados recebidos.
- **Autenticação e autorização:** implementar controle de acesso robusto.
- **Proteção de dados:** usar técnicas de criptografia para proteger dados sensíveis.
- **Logs e monitoramento:** registrar atividades para detecção de intrusões.

Introdução ao Spring Security

Introdução ao Spring Security

Programação Web II

O **Spring Security** é um framework para segurança de aplicações web, oferecendo autenticação, autorização e proteção contra ataques comuns como *CSRF* e *XSS*.

- **CSRF (Cross-Site Request Forgery):** Ataque que faz o usuário executar ações sem perceber, explorando sua sessão autenticada.
→ **Spring Security** protege usando tokens CSRF em cada requisição.
- **XSS (Cross-Site Scripting):** Ataque que injeta scripts maliciosos em páginas web para roubar dados ou manipular o conteúdo.
→ **Spring Security** previne com filtros e cabeçalhos de segurança.

Introdução ao Spring Security

Programação Web II

Configuração inicial:

→ pom.xml:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

→ build.gradle:

```
implementation 'org.springframework.boot:spring-boot-starter-security'
```


Configurando segurança básica:

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/publico").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin();
        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails user = User.withDefaultPasswordEncoder()
            .username("user")
            .password("password")
            .roles("USER")
            .build();
        return new InMemoryUserDetailsManager(user);
    }
}
```

Explicação das anotações

- **@Configuration:** define uma classe de configuração.
- **@Bean:** indica um método gerador de Bean.

Gerenciamento de Usuários em Banco de Dados

O **Spring Security** utiliza um mecanismo de autenticação que verifica as credenciais fornecidas por um usuário contra uma fonte de dados configurada (banco de dados, memória ou serviço externo).

Durante a autenticação, o framework valida as informações de login e, se bem-sucedido, cria uma sessão de usuário segura.

O processo envolve dois componentes principais:

- **UserDetails:** a interface representa as informações do usuário autenticado. Ela define métodos como **getUsername()**, **getPassword()** e **getAuthorities()**, que fornecem detalhes necessários para autenticação e autorização.
- **UserDetailsService:** essa interface é responsável por recuperar informações de usuários a partir de uma fonte de dados. O método **loadUserByUsername()** é chamado pelo *Spring Security* durante o processo de autenticação para buscar um usuário.

Introdução ao Spring Security

Programação Web II

Criando a entidade **Usuario**

```
@Entity
@Data
public class Usuario implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;
    private String password;
    private String role;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(() -> "ROLE_" + role);
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }
}
```


Configurando repositório

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {  
    Optional<Usuario> findByUsername(String username);  
}
```

Configurando o serviço e o UserDetailsService

```
@Service  
public class UsuarioService implements UserDetailsService {  
  
    private final UsuarioRepository usuarioRepository;  
  
    public UsuarioService(UsuarioRepository usuarioRepository) {  
        this.usuarioRepository = usuarioRepository;  
    }  
  
    @Override  
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {  
        return usuarioRepository.findByUsername(username)  
            .orElseThrow(() -> new UsernameNotFoundException("Usuário não encontrado: " + username));  
    }  
}
```

Configuração de segurança atualizada

A configuração de segurança deve ser aprimorada para incluir suporte a autenticação baseada em banco de dados e proteção de senhas com `BCryptPasswordEncoder`. Essa abordagem melhora a segurança das credenciais e possibilita um gerenciamento de usuários mais robusto.

O `PasswordEncoder` é um componente central para proteger senhas no *Spring Security*. Ele realiza a codificação das senhas antes de armazená-las no banco de dados, garantindo que, mesmo que uma senha seja exposta, não possa ser usada diretamente. A codificação torna as senhas ilegíveis e dificulta ataques de força bruta.

Configuração de segurança atualizada

O `BCryptPasswordEncoder` utiliza um algoritmo de hash robusto para proteger senhas. Ele aplica um processo conhecido como "salting", adicionando valores aleatórios a cada senha antes de codificá-la, tornando ataques de "*rainbow tables*" praticamente inviáveis. Além disso, o **BCrypt** é resistente a ataques de força bruta devido ao seu fator de trabalho ajustável, que aumenta a complexidade do cálculo do hash.

Introdução ao Spring Security

Programação Web II

Configuração de segurança atualizada

```
@Configuration
public class SecurityConfig {

    private final UserDetailsService userDetailsService;
    private final PasswordEncoder passwordEncoder;

    public SecurityConfig(UserDetailsService userDetailsService, PasswordEncoder passwordEncoder)
    {
        this.userDetailsService = userDetailsService;
        this.passwordEncoder = passwordEncoder;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/publico").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin();
        return http.build();
    }

    @Bean
    public AuthenticationManager authManager(HttpSecurity http) throws Exception {
        return http.getSharedObject(AuthenticationManagerBuilder.class)
            .userDetailsService(userDetailsService)
            .passwordEncoder(passwordEncoder)
            .and()
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

O que é Salting?

É o processo de adicionar dados aleatórios (chamados salt) à senha antes de aplicar o hash.

Isso garante que duas senhas iguais gerem hashes diferentes, impedindo comparações diretas.

```
Senha: 123456  
Salt: XyZ9  
Hash final: hash(123456XyZ9)
```

Mesmo que outro usuário também tenha “123456” como senha, o salt será diferente, gerando outro hash.

O que é Rainbow Tables?

São tabelas pré-calculadas de milhões de senhas e seus respectivos hashes – usadas por atacantes para tentar descobrir senhas rapidamente.

O salting torna essas tabelas inúteis, já que cada senha tem um *salt* único.

Controle de Autorização e Segurança em Métodos

Programação Web II

Uma aplicação segura não utiliza apenas autenticação para validar usuários, mas também implementa controles de autorização para regulamentar o acesso a diferentes recursos com base em permissões ou papéis (roles).

O Spring Security oferece mecanismos robustos para implementar controle de acesso em endpoints HTTP e também em métodos de negócios no backend.

Controle de Autorização e Segurança em Métodos

Programação Web II

Controle de autorização com Spring Security

O Spring Security permite definir regras de autorização diretamente na configuração de segurança.

Por exemplo, você pode especificar quais roles têm acesso a diferentes partes do sistema.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/publico").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .formLogin()
        .and()
        .csrf().disable(); // desativa CSRF, se necessário (não recomendado em produção sem cautela)
    return http.build();
}
```

Nesse exemplo:

- Endpoints que começam com **/api/publico** são acessíveis a todos, sem autenticação.
- Endpoints que começam com **/api/admin/**** exigem que o usuário tenha a role ADMIN.
- Qualquer outro requer autenticação do usuário.

Segurança em Métodos (Method Security)

Spring Security também suporta controle de autorização diretamente no nível dos métodos.

Para utilizá-lo, primeiro ative o suporte a segurança em métodos usando a anotação **@EnableMethodSecurity** na classe de configuração principal:

```
@Configuration
@EnableMethodSecurity
public class MethodSecurityConfig {
}
```

Com isso ativado, você pode usar anotações como **@PreAuthorize**, **@PostAuthorize**, **@Secured** e **@RolesAllowed** para restringir acesso a métodos específicos.

Segurança em Métodos (Method Security)

Exemplo com `@PreAuthorize`

```
@Service
public class DocumentoService {

    @PreAuthorize("hasRole('ADMIN')")
    public void excluirDocumento(Long id) {
        // Apenas usuários com a role ADMIN podem excluir documentos
        System.out.println("Documento excluído: " + id);
    }

    @PreAuthorize("hasRole('USER') or hasRole('ADMIN')")
    public void visualizarDocumento(Long id) {
        // Usuários com roles USER ou ADMIN podem visualizar documentos
        System.out.println("Documento visualizado: " + id);
    }
}
```

As expressões em **@PreAuthorize** são escritas em *SpEL* (*Spring Expression Language*) e permitem alta flexibilidade na definição de regras de autorização.

Exercícios

Exercício 1: Implementar o Spring Security

Escolha um projeto já desenvolvido em aula e implemente as configurações do Spring Security.

Incluindo:

- Autenticação de usuário e senha
- limitando as url que serão públicas e privadas

Obrigada